

Übungseinheit: Strukturierte Programme

Vom Skript zum sauberen Python-Programm

Dorian Zwanzig, Claude

8. Juni 2026

Inhaltsverzeichnis

1	Über diese Übung	2
2	Vorbereitung	3
3	Die wichtigste Regel: Die 5-Bausteine-Struktur	4
4	Teil 1 – Aufwärmen: Struktur anwenden	5
4.1	Aufgabe 1: Messreihen-Auswertung	5
4.2	Aufgabe 2: Wartungsplan an der Fertigungslinie	6
5	Teil 2 – Daten strukturiert verarbeiten	8
5.1	Aufgabe 3: Produktions-Balkendiagramm	8
5.2	Aufgabe 4: Robuste Eingabe-Validierung	9
6	Teil 3 – Ein eigenständiges Programm	11
6.1	Aufgabe 5: Warenwirtschaft mit Menü (Capstone)	11
6.2	Aufgabe 6 (Bonus): Rechnung mit Mengenrabatt	13
7	Musterlösungen	15
7.1	Lösung 1: statistik_messreihe.py	15
7.2	Lösung 2: wartungsplan.py	17
7.3	Lösung 3: produktions_balkendiagramm.py	19
7.4	Lösung 4: eingabe_validierung.py	20
7.5	Lösung 5: warenwirtschaft_cli.py	22
7.6	Lösung 6 (Bonus): rechnung_mengenrabatt.py	24
8	Abschluss-Reflexion	26

1 Über diese Übung

Lernziele

Nach Abschluss dieser Übung können Sie:

- **Ein loses Skript in ein strukturiertes Programm** mit Funktionen und `main()` überführen
- **Programme aus mehreren Hilfsfunktionen** aufbauen, von denen jede genau **eine Aufgabe** hat
- Die **5-Bausteine-Struktur** (Imports -> Konstanten -> Hilfsfunktionen -> `main()` -> Einstiegspunkt) sicher anwenden
- **Konstanten** sinnvoll einsetzen (GROSSBUCHSTABEN) und globale Variablen vermeiden
- `return` **statt** `print` in Berechnungsfunktionen verwenden
- **Robuste Benutzereingaben** mit `try/except` und Schleifen verarbeiten
- **Vollständige, lauffähige .py-Programme** schreiben, die ein sauberes Programm darstellen

Zeitbedarf: ca. 120 Minuten **Schwierigkeitsgrad:** Mittel (mit einer Herausforderung am Ende)

Voraussetzungen

Für diese Übung sollten Sie folgende Konzepte sicher beherrschen:

- **Variablen und einfache Datentypen** (Notebooks 04–05)
- **Listen, Tupel und Dictionaries** (Notebook 06)
- **Funktionen** mit Parametern und Rückgabewert (Notebook 07)
- **Operatoren** (Notebook 08)
- **Verzweigungen** mit `if/elif/else` und `match-case` (Notebook 09)
- **Schleifen** mit `for` und `while`, `break/continue` (Notebook 10)
- **Fehlerbehandlung** mit `try/except` (Notebook 11)

Diese Übung vertieft die Konzepte aus:

- **Notebook 15: Vom Skript zum strukturierten Programm**

Falls Sie unsicher sind, wiederholen Sie bitte das entsprechende Notebook.

Hinweis zur Bearbeitung

Dateiorganisation:

In dieser Übung schreiben Sie **echte Python-Programme**. Legen Sie für jede Aufgabe eine eigene `.py`-Datei an:

- `statistik_messreihe.py`
- `wartungsplan.py`
- `produktions_balkendiagramm.py`
- `eingabe_validierung.py`
- `warenwirtschaft_cli.py`
- `rechnung_mengenrabatt.py` (Bonus)

Jedes Programm soll **direkt lauffähig** sein: Öffnen Sie ein Terminal und starten Sie es mit `python statistik_messreihe.py`.

Hinweis zu `if __name__ == "__main__":`:

In dieser Übung beenden wir jedes Programm mit einem direkten `main()`-Aufruf am Dateiende. Für „echte“ wiederverwendbare `.py`-Module ergänzt man später `if __name__ == "__main__":` – das ist hier **nicht** Teil der Aufgabe.

2 Vorbereitung

Aufwand: ca. 8 Minuten

Technische Vorbereitung

1. Erstellen Sie einen neuen Ordner `uebung_programme`
2. Öffnen Sie Ihre Python-Entwicklungsumgebung (VS Code, PyCharm, etc.)
3. Erstellen Sie die erste Datei `umgebungstest.py`

Mentale Vorbereitung

In dieser Übung geht es **nicht** um neue Syntax, sondern um **sauberen Aufbau**. Sie kennen alle Bausteine bereits – heute lernen Sie, sie zu einem ordentlichen Programm zusammenzusetzen. Achten Sie bei jeder Aufgabe darauf:

1. **Eine Funktion – eine Aufgabe:** Wenn Sie „und“ sagen müssen, sind es zwei Funktionen.
2. **main() ist der Dirigent:** Sie ruft nur auf, sie rechnet nicht selbst.
3. **Berechnen heißt return, nicht print:** Nur Ausgabe-Funktionen drucken.

Test der Umgebung

Führen Sie folgenden Code in `umgebungstest.py` aus:

```
# Umgebungstest

def quadrat(zahl):
    """Gibt das Quadrat einer Zahl zurück."""
    return zahl * zahl

def main():
    """Testet die Programmstruktur."""
    print("[OK] Funktion und main() funktionieren")
    print(f"[OK] quadrat(5) = {quadrat(5)}")

main()
```

Erwartete Ausgabe:

```
[OK] Funktion und main() funktionieren
[OK] quadrat(5) = 25
```

Wenn das funktioniert, können Sie beginnen!

3 Die wichtigste Regel: Die 5-Bausteine-Struktur

Jedes Programm in dieser Übung folgt **derselben** Struktur:

```
# --- 1. Imports ---          Was brauche ich? (oft nichts)
# --- 2. Konstanten ---       Was ändert sich nie? (GROSSBUCHSTABEN)
# --- 3. Hilfsfunktionen ---   Was sind die Einzelschritte?
# --- 4. Hauptfunktion ---    def main(): Was ist der Gesamtablauf?
# --- 5. Einstiegspunkt ---    main()
```

Info-Box: Struktur-Checkliste

Prüfen Sie jedes Ihrer Programme gegen diese Liste:

- ☐ Imports stehen ganz oben (oder Kommentar „keine Imports nötig“)
- ☐ Konstanten stehen darunter, in GROSSBUCHSTABEN
- ☐ Jede Hilfsfunktion hat **eine** Aufgabe und einen **Docstring**
- ☐ Berechnungs-Funktionen nutzen `return`, nur Ausgabe-Funktionen `print`
- ☐ Funktionen nutzen **Parameter** statt globaler Variablen
- ☐ Es gibt **eine** `main()`-Funktion, die sich wie eine Zusammenfassung liest
- ☐ Am Ende steht der direkte Aufruf `main()`

4 Teil 1 – Aufwärmen: Struktur anwenden

Kompetenzstufe: Anwenden

4.1 Aufgabe 1: Messreihen-Auswertung

Schwierigkeit: Einfach · **Aufwand:** ca. 18 Minuten

Aufgabenstellung

An einer CNC-gefertigten Welle wird mit einem Tastkopf der Durchmesser an mehreren Stellen gemessen. Schreiben Sie ein strukturiertes Programm `statistik_messreihe.py`, das eine Messreihe einliest und auswertet.

Das Sollmaß beträgt **50,0 mm**, die zulässige Toleranz **± 0,05 mm**.

Das Programm soll:

1. So lange Messwerte (mm) einlesen, bis eine **leere Eingabe** erfolgt
2. Anzahl, kleinsten und größten Wert, Mittelwert und Spannweite berechnen
3. Zählen, wie viele Werte **innerhalb der Toleranz** liegen
4. Einen formatierten Bericht ausgeben

Verwenden Sie mindestens diese Funktionen: `messwerte_einlesen()`, `mittelwert(werte)`, `spannweite(werte)`, `anzahl_in_toleranz(werte, soll, toleranz)` und `bericht_ausgeben(...)`.

Erwartete Ausgabe (Eingabe: 50.02, 49.97, 50.05, 50.00, 49.93, 50.08, 49.99, 50.01):

```
Messwert in mm (leer = fertig): 50.02
Messwert in mm (leer = fertig): 49.97
... (weitere Werte) ...
Messwert in mm (leer = fertig):
=== MESSREIHEN-AUSWERTUNG ===
Anzahl Messwerte: 8
Kleinsten Wert: 49.93 mm
Größter Wert: 50.08 mm
Mittelwert: 50.01 mm
Spannweite: 0.15 mm
In Toleranz (50.0 +/- 0.05): 6 von 8
```

Hilfestellungen

Problemanalyse:

- Unbekannte Anzahl an Eingaben -> while-Schleife mit Abbruch bei leerer Eingabe
- „Innerhalb der Toleranz“ heißt: `abs(wert - soll) <= toleranz`
- Sollmaß und Toleranz ändern sich nicht -> **Konstanten**

Lösungsschritte:

1. Konstanten `SOLLMAS` = 50.0 und `TOLERANZ` = 0.05 definieren
2. `messwerte_einlesen()` schreiben: leere Liste, while True, bei "" -> break, sonst float(...) anhängen
3. Berechnungsfunktionen mit return schreiben (`mittelwert`, `spannweite`, `anzahl_in_toleranz`)
4. `bericht_ausgeben(...)` ruft die Berechnungen auf und druckt – die **einzigste** Funktion mit print
5. `main()`: einlesen -> Bericht ausgeben

Pseudocode:

KONSTANTE `SOLLMAS` = 50.0, `TOLERANZ` = 0.05

```
FUNKTION messwerte_einlesen
    werte = leere Liste
```

```

WIEDERHOLE
    eingabe = EINGABE
    WENN eingabe leer: BEENDE SCHLEIFE
    Hänge float(eingabe) an werte
GIB werte ZURÜCK

FUNKTION anzahl_in_toleranz(werte, soll, toleranz)
    zaehle Werte mit abs(wert - soll) <= toleranz
    GIB Anzahl ZURÜCK

FUNKTION main
    werte = messwerte_einlesen()
    bericht_ausgeben(werte, SOLLMASS, TOLERANZ)

```

4.2 Aufgabe 2: Wartungsplan an der Fertigungslinie

Schwierigkeit: Einfach–Mittel · **Aufwand:** ca. 16 Minuten

Aufgabenstellung

An einer Fertigungslinie wird jedes Teil fortlaufend nummeriert (1, 2, 3, ...). Nach festen Intervallen ist eine Wartung fällig:

- Jedes **3.** Teil -> **Sichtprüfung**
- Jedes **5.** Teil -> **Schmierung**
- Teile, die durch **3 und 5** teilbar sind -> **Komplettwartung**

Schreiben Sie `wartungsplan.py`. Das Programm soll:

1. Die Anzahl der Teile vom Benutzer abfragen
2. Für jedes Teil ausgeben, welche Wartung fällig ist (oder –)
3. Am Ende zusammenfassen, wie oft jede Wartungsart vorkam

Schreiben Sie eine reine Logik-Funktion `wartung_fuer_position(nummer)`, die das passende Wartungs-Label **zurückgibt** (nicht druckt!).

Erwartete Ausgabe (Eingabe: 15):

```

Anzahl der Teile: 15
=== WARTUNGSPLAN (15 Teile) ===
Teil 1: -
Teil 2: -
Teil 3: Sichtprüfung
Teil 4: -
Teil 5: Schmierung
... (weitere Teile) ...
Teil 15: Komplettwartung
--- Zusammenfassung ---
Sichtprüfung: 4
Schmierung: 2
Komplettwartung: 1

```

Hilfestellungen

Problemanalyse:

- **Klassische Falle:** Der „beides“-Fall (durch 3 **und** 5) muss **zuerst** geprüft werden – sonst wird er nie erreicht!
- Modulo % prüft Teilbarkeit: `nummer % 3 == 0`
- Häufigkeiten zählen -> ein Dictionary als Zähler

Lösungsschritte:

1. `wartung_fuer_position(nummer)`: zuerst `nummer % 3 == 0 and nummer % 5 == 0`, dann die Einzelfälle, sonst "-" zurückgeben
2. `wartungsplan_ausgeben(anzahl)`: for `nummer in range(1, anzahl + 1)`, Label holen und drucken
3. `wartungen_zaehlen(anzahl)`: Zähler-Dictionary mit drei Schlüsseln, hochzählen
4. `zusammenfassung_ausgeben(zaehler)`: über `zaehler.items()` drucken
5. `main()`: Anzahl einlesen -> Plan -> Zähler -> Zusammenfassung

Pseudocode:

```
KONSTANTE SICHT_INTERVALL = 3, SCHMIER_INTERVALL = 5
```

```
FUNKTION wartung_fuer_position(nummer)
```

```
    WENN nummer durch 3 UND durch 5 teilbar: GIB "Komplettwartung"  
    SONST WENN durch 3 teilbar: GIB "Sichtprüfung"  
    SONST WENN durch 5 teilbar: GIB "Schmierung"  
    SONST: GIB "-"
```

```
FUNKTION wartungsplan_ausgeben(anzahl)
```

```
    FÜR nummer VON 1 BIS anzahl  
        DRUCKE nummer und wartung_fuer_position(nummer)
```

```
FUNKTION wartungen_zaehlen(anzahl)
```

```
    zaehler = {"Sichtprüfung": 0, "Schmierung": 0, "Komplettwartung": 0}  
    FÜR nummer VON 1 BIS anzahl  
        wartung = wartung_fuer_position(nummer)  
        WENN wartung in zaehler: erhöhe zaehler[wartung] um 1  
    GIB zaehler ZURÜCK
```

```
FUNKTION zusammenfassung_ausgeben(zaehler)
```

```
    FÜR JEDE art, anzahl IN zaehler  
        DRUCKE art und anzahl
```

```
FUNKTION main
```

```
    anzahl = EINGABE (Ganzzahl)  
    DRUCKE Überschrift  
    wartungsplan_ausgeben(anzahl)  
    zaehler = wartungen_zaehlen(anzahl)  
    zusammenfassung_ausgeben(zaehler)
```

5 Teil 2 – Daten strukturiert verarbeiten

Kompetenzstufe: Analysieren

5.1 Aufgabe 3: Produktions-Balkendiagramm

Schwierigkeit: Mittel · **Aufwand:** ca. 18 Minuten

Aufgabenstellung

Die Tagesstückzahlen einer Produktionswoche sollen als **horizontales Balkendiagramm** aus ASCII-Zeichen dargestellt werden – pro Wochentag ein liegender Balken. Ein Stern (*) steht für 10 produzierte Stück.

Schreiben Sie `produktions_balkendiagramm.py`. Das Programm soll:

1. Für jeden Wochentag (Mo–Fr) die Tagesstückzahl einlesen
2. Pro Tag einen waagerechten Balken aus *-Zeichen zeichnen
3. Den Spitzentag (höchste Produktion) ermitteln und melden

Schreiben Sie eine Funktion `balken_erzeugen(menge, pro_stern)`, die den Balken-String **zurückgibt**.

Erwartete Ausgabe (Eingabe: 120, 95, 140, 110, 80):

```
Produktion Mo: 120
Produktion Di: 95
Produktion Mi: 140
Produktion Do: 110
Produktion Fr: 80
=== PRODUKTIONS-BALKENDIAGRAMM (1 * = 10 Stück) ===
Mo | ***** 120
Di | ***** 95
Mi | ***** 140
Do | ***** 110
Fr | ***** 80
Spitzentag: Mi mit 140 Stück
```

Hilfestellungen

Problemanalyse:

- Ein String lässt sich mit * vervielfachen: "*" * 5 ergibt "*****"
- Anzahl Sterne = `menge // pro_stern` (Ganzzahldivision)
- Daten Tag -> Menge -> ein Dictionary

Lösungsschritte:

1. Konstanten `WOCHENTAGE = ["Mo", "Di", "Mi", "Do", "Fr"]` und `EINHEITEN_PRO_STERN = 10`
2. `produktion_einlesen(tage)`: leeres Dictionary, for tag in tage -> `int(input(...))` speichern
3. `balken_erzeugen(menge, pro_stern)`: `return "*" * (menge // pro_stern)`
4. `balkendiagramm_ausgeben(daten, pro_stern)`: über `daten.items()` Balken erzeugen und drucken
5. `spitzentag_finden(daten)`: höchste Menge per Schleife suchen, (tag, menge) zurückgeben

Pseudocode:

```
KONSTANTE WOCHENTAGE = ["Mo", "Di", "Mi", "Do", "Fr"], EINHEITEN_PRO_STERN = 10
```

```
FUNKTION produktion_einlesen(tage)
    daten = leeres Dictionary
    FÜR JEDEN tag IN tage
        daten[tag] = EINGABE (Ganzzahl)
    GIB daten ZURÜCK
```

```

FUNKTION balken_erzeugen(menge, pro_stern)
    GIB "*" mal (menge // pro_stern) ZURÜCK

FUNKTION balkendiagramm_ausgeben(daten, pro_stern)
    FÜR JEDEN tag, menge IN daten
        balken = balken_erzeugen(menge, pro_stern)
        DRUCKE tag, balken und menge

FUNKTION spitzentag_finden(daten)
    hoechste = -1, spitzentag = ""
    FÜR JEDEN tag, menge IN daten
        WENN menge > hoechste: merke tag und menge
    GIB (spitzentag, hoechste) ZURÜCK

FUNKTION main
    daten = produktion_einlesen(WOCHENTAGE)
    DRUCKE Überschrift
    balkendiagramm_ausgeben(daten, EINHEITEN_PRO_STERN)
    tag, menge = spitzentag_finden(daten)
    DRUCKE "Spitzentag:" tag und menge

```

5.2 Aufgabe 4: Robuste Eingabe-Validierung

Schwierigkeit: Mittel · **Aufwand:** ca. 20 Minuten

Aufgabenstellung

In der Qualitätskontrolle dürfen nur **gültige Zahlen** in einem erlaubten Bereich eingegeben werden. Tippt der Benutzer Unsinn (z. B. Buchstaben) oder einen Wert außerhalb des Bereichs, soll erneut gefragt werden.

Schreiben Sie `eingabe_validierung.py`. Das Programm soll:

1. Über eine **wiederverwendbare** Funktion `gueltige_zahl_einlesen(prompt, minimum, maximum)` zwei Werte einlesen: produzierte Teile (mind. 1) und Ausschuss-Teile (0 bis „produziert“)
2. Die Ausschussquote in Prozent berechnen
3. Die Quote bewerten: $< 2\%$ „i.O.“, $2\text{--}5\%$ „beobachten“, $\geq 5\%$ „kritisch“
4. Einen Bericht ausgeben

Info-Box: `try/except` für robuste Eingaben

```

try:
    zahl = float(eingabe)
except ValueError:
    print("Das war keine Zahl!")

```

So stürzt das Programm bei einer Falscheingabe nicht ab.

Erwartete Ausgabe (Eingabe: abc, -5, 1000, 15):

```

Produzierte Teile: abc
Ungültig: Bitte eine Zahl eingeben.
Produzierte Teile: -5
Ungültig: Wert muss zwischen 1 und 100000 liegen.
Produzierte Teile: 1000
Ausschuss-Teile: 15
=== QUALITÄTSBERICHT ===
Produziert: 1000 | Ausschuss: 15

```

Ausschussquote: 1.50 %
 Status: i.O.

Hilfestellungen

Problemanalyse:

- Zwei Fehlerquellen: keine Zahl (try/except) **und** Zahl außerhalb des Bereichs (if)
- Wiederholen, bis gültig -> while True mit return bei Erfolg
- gueltige_zahl_einlesen ist die **Kernfunktion** – sie wird auch in späteren Programmen nützlich!

Lösungsschritte:

1. gueltige_zahl_einlesen(prompt, minimum, maximum): while True, input, try: float(...) except ValueError: continue, dann Bereich prüfen, bei Erfolg return
2. ausschussquote(produziert, ausschuss): return (ausschuss / produziert) * 100
3. quote_bewerten(quote): mit if/elif/else einen Status-String zurückgeben
4. ergebnis_ausgeben(...): Bericht drucken
5. main(): zwei Werte einlesen -> Quote -> Bewertung -> Ausgabe

Pseudocode:

KONSTANTE GRENZE_BEOBACHTEN = 2.0, GRENZE_KRITISCH = 5.0

FUNKTION gueltige_zahl_einlesen(prompt, minimum, maximum)

 WIEDERHOLE

 eingabe = EINGABE(prompt)

 VERSUCHE zahl = float(eingabe)

 BEI FEHLER: melde "keine Zahl", WEITER

 WENN zahl < minimum ODER zahl > maximum: melde Bereichsfehler, WEITER

 GIB zahl ZURÜCK

FUNKTION ausschussquote(produziert, ausschuss)

 GIB (ausschuss / produziert) * 100 ZURÜCK

FUNKTION quote_bewerten(quote)

 WENN quote < GRENZE_BEOBACHTEN: GIB "i.O."

 SONST WENN quote < GRENZE_KRITISCH: GIB "beobachten"

 SONST: GIB "kritisch"

FUNKTION ergebnis_ausgeben(produziert, ausschuss, quote, status)

 DRUCKE Bericht mit produziert, ausschuss, quote und status

FUNKTION main

 produziert = gueltige_zahl_einlesen("Produzierte Teile: ", 1, 100000)

 ausschuss = gueltige_zahl_einlesen("Ausschuss-Teile: ", 0, produziert)

 quote = ausschussquote(produziert, ausschuss)

 status = quote_bewerten(quote)

 ergebnis_ausgeben(produziert, ausschuss, quote, status)

6 Teil 3 – Ein eigenständiges Programm

Kompetenzstufe: Erschaffen

6.1 Aufgabe 5: Warenwirtschaft mit Menü (Capstone)

Schwierigkeit: Herausforderung · **Aufwand:** ca. 28 Minuten

Aufgabenstellung

Schreiben Sie ein **menügesteuertes** Auswertungs-Programm `warenwirtschaft_cli.py` für ein kleines Lager. Das Programm zeigt wiederholt ein Menü und führt die gewählte Aktion aus, bis der Benutzer „Beenden“ wählt.

Das Lager ist als Dictionary gegeben, wobei jeder Wert ein **Tupel** (bestand, einzelpreis) ist:

```
{
    "Schraube M8": (150, 0.05),
    "Mutter M8": (200, 0.03),
    "Bolzen M12": (80, 0.15),
    "Dübel 8mm": (500, 0.02),
}
```

Das Menü bietet:

1. **Bestand anzeigen** – alle Artikel mit Bestand und Preis
2. **Artikel suchen** – Name eingeben, Bestand/Preis/Wert ausgeben (oder „nicht gefunden“)
3. **Gesamtwert berechnen** – Summe aus bestand * preis über alle Artikel
4. **Niedrigbestand-Warnung** – alle Artikel unter dem Mindestbestand (100)
5. **Beenden**

Info-Box: Menüschleife mit match-case

```
while True:
    menue_anzeigen()
    wahl = input("Auswahl: ")
    match wahl:
        case "1":
            ...
        case "5":
            break
```

Tipp: Lagern Sie die Auswahl-Logik in eine eigene Funktion `auswahl_verarbeiten(lager, wahl)` aus, die False zurückgibt, wenn beendet werden soll.

Erwartete Ausgabe (Auswahl: 3, dann 4, dann 5):

```
=== WARENWIRTSCHAFT ===
```

```
1) Bestand anzeigen
2) Artikel suchen
3) Gesamtwert berechnen
4) Niedrigbestand-Warnung
5) Beenden
Auswahl: 3
Gesamtwert: 35.50 €
```

```
=== WARENWIRTSCHAFT ===
```

```
... (Menü erneut) ...
Auswahl: 4
[WARNUNG] Unter Mindestbestand (100): Bolzen M12
```

```
=== WARENWIRTSCHAFT ===
```

```
... (Menü erneut) ...
Auswahl: 5
Programm beendet.
```

Hilfestellungen

Problemanalyse:

- Endlose Wiederholung bis Abbruch -> while True mit break
- Tupel im Dictionary entpacken: for name, (bestand, preis) in lager.items()
- Such-Funktion soll None **zurückgeben**, wenn nicht gefunden – der Aufrufer entscheidet über die Meldung

Lösungsschritte:

1. lager_erstellen() gibt das Dictionary zurück; Konstante MINDESTBESTAND = 100
2. menue_anzeigen() druckt die fünf Optionen
3. bestand_anzeigen(lager), artikel_suchen(lager, name), gesamtwert(lager), niedrigbestand(lager, grenze) als getrennte Funktionen
4. auswahl_verarbeiten(lager, wahl): match wahl ruft die passende Funktion auf; case "5" -> return False
5. main(): Lager erstellen, while True: Menü -> Eingabe -> if not auswahl_verarbeiten(...): break

Pseudocode:

```
KONSTANTE MINDESTBESTAND = 100
```

```
FUNKTION lager_erstellen()
    GIB Dictionary {name: (bestand, preis), ...} ZURÜCK
```

```
FUNKTION menue_anzeigen()
    DRUCKE die fünf Menüoptionen
```

```
FUNKTION bestand_anzeigen(lager)
    FÜR JEDEN name, (bestand, preis) IN lager
        DRUCKE name, bestand und preis
```

```
FUNKTION artikel_suchen(lager, name)
    WENN name in lager: GIB lager[name] ZURÜCK
    SONST: GIB None ZURÜCK
```

```
FUNKTION gesamtwert(lager)
    summe = 0
    FÜR JEDEN bestand, preis IN lager-Werten
        summe = summe + bestand * preis
    GIB summe ZURÜCK
```

```
FUNKTION niedrigbestand(lager, grenze)
    namen = leere Liste
    FÜR JEDEN name, (bestand, preis) IN lager
        WENN bestand < grenze: hänge name an namen
    GIB namen ZURÜCK
```

```
FUNKTION auswahl_verarbeiten(lager, wahl)
    VERGLEICHE wahl MIT
    "1": bestand_anzeigen(lager)
    "2": name einlesen, artikel_suchen(...), Treffer oder Fehlmeldung drucken
    "3": DRUCKE gesamtwert(lager)
    "4": DRUCKE niedrigbestand(lager, MINDESTBESTAND) als Warnung
    "5": DRUCKE "Programm beendet.", GIB False ZURÜCK
```

```

        sonst: DRUCKE "Ungültige Auswahl"
    GIB True ZURÜCK

FUNKTION main
    lager = lager_erstellen()
    WIEDERHOLE
        menue_anzeigen()
        wahl = EINGABE
        WENN NICHT auswahl_verarbeiten(lager, wahl): BEENDE SCHLEIFE

```

6.2 Aufgabe 6 (Bonus): Rechnung mit Mengenrabatt

Schwierigkeit: Herausforderung · **Aufwand:** ca. 14 Minuten · *Falls Zeit bleibt!*

Aufgabenstellung

Schreiben Sie `rechnung_mengenrabatt.py`. Der Benutzer erfasst Bestellpositionen (Name, Menge, Einzelpreis), bis er „fertig“ eingibt. Anschließend wird eine Rechnung mit gestaffeltem **Mengenrabatt** und **19 % MwSt** erstellt.

Rabattstaffel je Position (nach Menge):

- **< 100 Stück:** 0 %
- **100–499 Stück:** 5 %
- **≥ 500 Stück:** 10 %

Erwartete Ausgabe (Eingaben: Schraube M8 / 250 / 0.05, Bolzen M12 / 40 / 0.15, Dübel 8mm / 600 / 0.02, fertig):

```

=== RECHNUNG ===
Schraube M8      250 x 0.05 € - 5 % =    11.88 €
Bolzen M12       40 x 0.15 € - 0 % =     6.00 €
Dübel 8mm       600 x 0.02 € -10 % =    10.80 €
-----
Netto:          28.68 €
MwSt (19 %):    5.45 €
Brutto:         34.12 €

```

Hilfestellungen

Lösungsschritte:

1. `bestellung_einlesen()`: while True, bei "fertig" (klein geschrieben) -> break, sonst Name, int(menge), float(preis) als Tupel anhängen
2. `rabattsatz(menge)`: mit if/elif/else 0.0, 0.05 oder 0.10 zurückgeben
3. `positionspreis(menge, einzelpreis)`: $\text{menge} * \text{einzelpreis} * (1 - \text{rabattsatz(menge)})$
4. `nettosumme(bestellung)`: über alle Positionen summieren
5. `rechnung_ausgeben(bestellung, mwst_satz)`: Positionen drucken, Netto/MwSt/Brutto berechnen und drucken

Pseudocode:

KONSTANTE MWST_SATZ = 0.19

```

FUNKTION bestellung_einlesen()
    positionen = leere Liste
    WIEDERHOLE
        name = EINGABE
        WENN name == "fertig": BEENDE SCHLEIFE
        menge = EINGABE (Ganzzahl), einzelpreis = EINGABE (Dezimalzahl)

```

```
        hänge (name, menge, einzelpreis) an positionen
    GIB positionen ZURÜCK

FUNKTION rabattsatz(menge)
    WENN menge >= 500: GIB 0.10
    SONST WENN menge >= 100: GIB 0.05
    SONST: GIB 0.0

FUNKTION positionspreis(menge, einzelpreis)
    GIB menge * einzelpreis * (1 - rabattsatz(menge)) ZURÜCK

FUNKTION nettosumme(bestellung)
    summe = 0
    FÜR JEDE (name, menge, einzelpreis) IN bestellung
        summe = summe + positionspreis(menge, einzelpreis)
    GIB summe ZURÜCK

FUNKTION rechnung_ausgeben(bestellung, mwst_satz)
    FÜR JEDE Position: DRUCKE Zeile mit Rabatt und positionspreis(...)
    netto = nettosumme(bestellung)
    mwst = netto * mwst_satz
    DRUCKE netto, mwst und (netto + mwst)

FUNKTION main
    bestellung = bestellung_einlesen()
    rechnung_ausgeben(bestellung, MWST_SATZ)
```

7 Musterlösungen

Vergleichen Sie **erst nach** eigenem Versuch. Jede Lösung folgt der 5-Bausteine-Struktur.

7.1 Lösung 1: statistik_messreihe.py

```
# statistik_messreihe.py - Auswertung einer Messreihe

# --- 1. Imports ---
# keine Imports nötig

# --- 2. Konstanten ---
SOLLMASS = 50.0
TOLERANZ = 0.05

# --- 3. Hilfsfunktionen ---
def messwerte_einlesen():
    """Liest Messwerte ein, bis eine leere Eingabe erfolgt."""
    werte = []
    while True:
        eingabe = input("Messwert in mm (leer = fertig): ")
        if eingabe == "":
            break
        werte.append(float(eingabe))
    return werte

def mittelwert(werte):
    """Berechnet den arithmetischen Mittelwert."""
    summe = 0.0
    for wert in werte:
        summe = summe + wert
    return summe / len(werte)

def spannweite(werte):
    """Berechnet die Differenz zwischen größtem und kleinstem Wert."""
    return max(werte) - min(werte)

def anzahl_in_toleranz(werte, soll, toleranz):
    """Zählt die Werte, die innerhalb der Toleranz liegen."""
    anzahl = 0
    for wert in werte:
        if abs(wert - soll) <= toleranz:
            anzahl = anzahl + 1
    return anzahl

def bericht_ausgeben(werte, soll, toleranz):
    """Gibt den vollständigen Auswertungsbericht aus."""
    print("=== MESSREIHEN-AUSWERTUNG ===")
    print(f"Anzahl Messwerte: {len(werte)}")
    print(f"Kleinster Wert: {min(werte):.2f} mm")
    print(f"Größter Wert: {max(werte):.2f} mm")
    print(f"Mittelwert: {mittelwert(werte):.2f} mm")
    print(f"Spannweite: {spannweite(werte):.2f} mm")
```

```
in_tol = anzahl_in_toleranz(werte, soll, toleranz)
print(f"In Toleranz ({soll} +/- {toleranz}): {in_tol} von {len(werte)}")

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Messreihe einlesen und auswerten."""
    werte = messwerte_einlesen()
    if not werte:
        print("Keine Messwerte eingegeben.")
        return
    bericht_ausgeben(werte, SOLLMASS, TOLERANZ)

# --- 5. Einstiegspunkt ---
main()
```

7.2 Lösung 2: wartungsplan.py

```
# wartungsplan.py - Wartungsplan entlang einer Fertigungslinie

# --- 1. Imports ---
# keine Imports nötig

# --- 2. Konstanten ---
SICHT_INTERVALL = 3
SCHMIER_INTERVALL = 5

# --- 3. Hilfsfunktionen ---
def wartung_fuer_position(nummer):
    """Gibt die fällige Wartung für eine Teil-Position zurück."""
    if nummer % SICHT_INTERVALL == 0 and nummer % SCHMIER_INTERVALL == 0:
        return "Komplettwartung"
    elif nummer % SICHT_INTERVALL == 0:
        return "Sichtprüfung"
    elif nummer % SCHMIER_INTERVALL == 0:
        return "Schmierung"
    else:
        return "-"

def wartungsplan_ausgeben(anzahl):
    """Gibt für jede Position die fällige Wartung aus."""
    for nummer in range(1, anzahl + 1):
        print(f"Teil {nummer:2d}: {wartung_fuer_position(nummer)}")

def wartungen_zaehlen(anzahl):
    """Zählt, wie oft jede Wartungsart vorkommt."""
    zaehler = {"Sichtprüfung": 0, "Schmierung": 0, "Komplettwartung": 0}
    for nummer in range(1, anzahl + 1):
        wartung = wartung_fuer_position(nummer)
        if wartung in zaehler:
            zaehler[wartung] = zaehler[wartung] + 1
    return zaehler

def zusammenfassung_ausgeben(zaehler):
    """Gibt die Häufigkeit jeder Wartungsart aus."""
    print("---- Zusammenfassung ----")
    for art, anzahl in zaehler.items():
        print(f"{art}: {anzahl}")

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Wartungsplan erstellen und zusammenfassen."""
    anzahl = int(input("Anzahl der Teile: "))
    print(f"=== WARTUNGSPLAN ({anzahl} Teile) ===")
    wartungsplan_ausgeben(anzahl)
    zaehler = wartungen_zaehlen(anzahl)
    zusammenfassung_ausgeben(zaehler)

# --- 5. Einstiegspunkt ---
```

```
main()
```

7.3 Lösung 3: produktions_balkendiagramm.py

```
# produktions_balkendiagramm.py - horizontales ASCII-Balkendiagramm der Tagesproduktion

# --- 1. Imports ---
# keine Imports nötig

# --- 2. Konstanten ---
WOCHENTAGE = ["Mo", "Di", "Mi", "Do", "Fr"]
EINHEITEN_PRO_STERN = 10

# --- 3. Hilfsfunktionen ---
def produktion_einlesen(tage):
    """Liest für jeden Wochentag die Tagesstückzahl ein."""
    daten = {}
    for tag in tage:
        daten[tag] = int(input(f"Produktion {tag}: "))
    return daten

def balken_erzeugen(menge, pro_stern):
    """Erzeugt einen Balken aus Sternen für eine Menge."""
    return "*" * (menge // pro_stern)

def balkendiagramm_ausgeben(daten, pro_stern):
    """Gibt das Balkendiagramm Zeile für Zeile aus."""
    for tag, menge in daten.items():
        balken = balken_erzeugen(menge, pro_stern)
        print(f"{tag:<3}| {balken} {menge}")

def spitzentag_finden(daten):
    """Findet den Tag mit der höchsten Produktion."""
    spitzentag = ""
    hoechste = -1
    for tag, menge in daten.items():
        if menge > hoechste:
            hoechste = menge
            spitzentag = tag
    return (spitzentag, hoechste)

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Produktionsdaten erfassen und visualisieren."""
    daten = produktion_einlesen(WOCHENTAGE)
    print(f"=== PRODUKTIONS-BALKENDIAGRAMM (1 * = {EINHEITEN_PRO_STERN} Stück) ===")
    balkendiagramm_ausgeben(daten, EINHEITEN_PRO_STERN)
    tag, menge = spitzentag_finden(daten)
    print(f"Spitzentag: {tag} mit {menge} Stück")

# --- 5. Einstiegspunkt ---
main()
```

7.4 Lösung 4: eingabe_validierung.py

```
# eingabe_validierung.py - Robuste Eingabe-Validierung

# --- 1. Imports ---
# keine Imports nötig

# --- 2. Konstanten ---
GRENZE_BEOBACHTEN = 2.0
GRENZE_KRITISCH = 5.0

# --- 3. Hilfsfunktionen ---
def gueltige_zahl_einlesen(prompt, minimum, maximum):
    """Liest eine Zahl ein und wiederholt bei ungültiger Eingabe."""
    while True:
        eingabe = input(prompt)
        try:
            zahl = float(eingabe)
        except ValueError:
            print("Ungültig: Bitte eine Zahl eingeben.")
            continue
        if zahl < minimum or zahl > maximum:
            print(f"Ungültig: Wert muss zwischen {minimum} und {maximum} liegen.")
            continue
        return zahl

def ausschussquote(produziert, ausschuss):
    """Berechnet die Ausschussquote in Prozent."""
    return (ausschuss / produziert) * 100

def quote_bewerten(quote):
    """Bewertet die Ausschussquote als Klartext-Status."""
    if quote < GRENZE_BEOBACHTEN:
        return "i.O."
    elif quote < GRENZE_KRITISCH:
        return "beobachten"
    else:
        return "kritisch"

def ergebnis_ausgeben(produziert, ausschuss, quote, status):
    """Gibt den Qualitätsbericht aus."""
    print("=== QUALITÄTSBERICHT ===")
    print(f"Produziert: {int(produziert)} | Ausschuss: {int(ausschuss)}")
    print(f"Ausschussquote: {quote:.2f} %")
    print(f>Status: {status}")

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Eingaben prüfen und Ausschussquote bewerten."""
    produziert = gueltige_zahl_einlesen("Produzierte Teile: ", 1, 100000)
    ausschuss = gueltige_zahl_einlesen("Ausschuss-Teile: ", 0, produziert)
    quote = ausschussquote(produziert, ausschuss)
    status = quote_bewerten(quote)
    ergebnis_ausgeben(produziert, ausschuss, quote, status)
```

```
# --- 5. Einstiegspunkt ---  
main()
```

7.5 Lösung 5: warenwirtschaft_cli.py

warenwirtschaft_cli.py - Menügesteuerte Warenwirtschaft (Auswertung)

--- 1. Imports ---

keine Imports nötig

--- 2. Konstanten ---

MINDESTBESTAND = 100

--- 3. Hilfsfunktionen ---

```
def lager_erstellen():
    """Erstellt ein Beispiel-Lager: Name -> (Bestand, Einzelpreis)."""
    return {
        "Schraube M8": (150, 0.05),
        "Mutter M8": (200, 0.03),
        "Bolzen M12": (80, 0.15),
        "Dübel 8mm": (500, 0.02),
    }
```

```
def menue_anzeigen():
    """Gibt das Auswahlmenü aus."""
    print("\n=== WARENWIRTSCHAFT ===")
    print("1) Bestand anzeigen")
    print("2) Artikel suchen")
    print("3) Gesamtwert berechnen")
    print("4) Niedrigbestand-Warnung")
    print("5) Beenden")
```

```
def bestand_anzeigen(lager):
    """Gibt alle Artikel mit Bestand und Preis aus."""
    for name, (bestand, preis) in lager.items():
        print(f"{name:<14} {bestand:>4} Stück {preis:.2f} €")
```

```
def artikel_suchen(lager, name):
    """Gibt (Bestand, Preis) eines Artikels zurück oder None."""
    if name in lager:
        return lager[name]
    return None
```

```
def gesamtwert(lager):
    """Berechnet den Gesamtwert des Lagers."""
    summe = 0.0
    for bestand, preis in lager.values():
        summe = summe + bestand * preis
    return summe
```

```
def niedrigbestand(lager, grenze):
    """Gibt die Namen aller Artikel unter der Grenze zurück."""
    namen = []
    for name, (bestand, preis) in lager.items():
        if bestand < grenze:
            namen.append(name)
```

```

    return namen

def auswahl_verarbeiten(lager, wahl):
    """Führt die gewählte Menüaktion aus; gibt False bei 'Beenden'."""
    match wahl:
        case "1":
            bestand_anzeigen(lager)
        case "2":
            name = input("Artikelname: ")
            treffer = artikel_suchen(lager, name)
            if treffer is None:
                print("[WARNUNG] Artikel nicht gefunden.")
            else:
                bestand, preis = treffer
                print(f"{name}: {bestand} Stück à {preis:.2f} € = {bestand * preis:.2f} €")
        case "3":
            print(f"Gesamtwert: {gesamtwert(lager):.2f} €")
        case "4":
            namen = niedrigbestand(lager, MINDESTBESTAND)
            if namen:
                print(f"[WARNUNG] Unter Mindestbestand ({MINDESTBESTAND}): {' '.join(namen)}")
            else:
                print("[OK] Alle Artikel über Mindestbestand.")
        case "5":
            print("Programm beendet.")
            return False
        case _:
            print("[WARNUNG] Ungültige Auswahl.")
    return True

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Menüsleife der Warenwirtschaft."""
    lager = lager_erstellen()
    while True:
        menue_anzeigen()
        wahl = input("Auswahl: ")
        if not auswahl_verarbeiten(lager, wahl):
            break

# --- 5. Einstiegspunkt ---
main()

```

7.6 Lösung 6 (Bonus): rechnung_mengenrabatt.py

```
# rechnung_mengenrabatt.py - Rechnung mit Mengenrabatt und MwSt
```

```
# --- 1. Imports ---
```

```
# keine Imports nötig
```

```
# --- 2. Konstanten ---
```

```
MWST_SATZ = 0.19
```

```
# --- 3. Hilfsfunktionen ---
```

```
def bestellung_einlesen():
```

```
    """Liest Bestellpositionen ein, bis 'fertig' eingegeben wird."""
```

```
    positionen = []
```

```
    while True:
```

```
        name = input("Artikelname (oder 'fertig'): ")
```

```
        if name.lower() == "fertig":
```

```
            break
```

```
        menge = int(input("Menge: "))
```

```
        einzelpreis = float(input("Einzelpreis in €: "))
```

```
        positionen.append((name, menge, einzelpreis))
```

```
    return positionen
```

```
def rabattsatz(menge):
```

```
    """Gibt den Rabattsatz abhängig von der Bestellmenge zurück."""
```

```
    if menge >= 500:
```

```
        return 0.10
```

```
    elif menge >= 100:
```

```
        return 0.05
```

```
    else:
```

```
        return 0.0
```

```
def positionspreis(menge, einzelpreis):
```

```
    """Berechnet den rabattierten Preis einer Position."""
```

```
    return menge * einzelpreis * (1 - rabattsatz(menge))
```

```
def nettosumme(bestellung):
```

```
    """Berechnet die Nettosumme aller Positionen."""
```

```
    summe = 0.0
```

```
    for name, menge, einzelpreis in bestellung:
```

```
        summe = summe + positionspreis(menge, einzelpreis)
```

```
    return summe
```

```
def rechnung_ausgeben(bestellung, mwst_satz):
```

```
    """Gibt die vollständige Rechnung aus."""
```

```
    print("=== RECHNUNG ===")
```

```
    for name, menge, einzelpreis in bestellung:
```

```
        rabatt = rabattsatz(menge)
```

```
        preis = positionspreis(menge, einzelpreis)
```

```
        print(f"{name:<14} {menge:>4} x {einzelpreis:~.2f} € -{int(rabatt * 100):>2} % = {preis:>8.2f}")
```

```
    netto = nettosumme(bestellung)
```

```
    mwst = netto * mwst_satz
```

```
    print("-" * 44)
```

```
    print(f"Netto: {netto:>8.2f} €")
```

```
print(f"MwSt ({int(mwst_satz * 100)} %): {mwst:>6.2f} €")
print(f"Brutto: {netto + mwst:>8.2f} €")

# --- 4. Hauptfunktion ---
def main():
    """Hauptprogramm: Bestellung erfassen und Rechnung erstellen."""
    bestellung = bestellung_einlesen()
    if not bestellung:
        print("Keine Positionen erfasst.")
        return
    rechnung_ausgeben(bestellung, MWST_SATZ)

# --- 5. Einstiegspunkt ---
main()
```

8 Abschluss-Reflexion

Sie haben sechs vollständige Programme geschrieben. Prüfen Sie zum Schluss:

- Liest sich jede `main()` wie eine **kurze Zusammenfassung** des Programms?
- Hat **jede** Hilfsfunktion genau **eine** Aufgabe und einen Docstring?
- Geben Ihre Berechnungs-Funktionen Werte mit `return` zurück, statt sie zu drucken?
- Stehen Ihre Konstanten in **GROSSBUCHSTABEN** ganz oben?

Merke: Gute Struktur ist kein Selbstzweck. Ein sauber aufgebautes Programm ist leichter zu lesen, zu testen, zu erweitern und im Team zu nutzen – genau das, was im Berufsalltag zählt.